

Analisis de redes de transporte reales

Estructuras de datos I

Jairo Ariza

Miguel Alvarez Orozco

Emmanuel Montoya

Fecha: 08/05/2026

1. Introducción

Cuando pensamos en grafos, en la primera aplicación que se me ocurre es las redes de transporte. Cada vez que abre Waze, Google Maps, o aplicaciones mas complejas como Uber, hay un algoritmo de camino mínimo en el backend de estas aplicaciones.

El dataset propuesto por el profesor es roadNet-PA, que es la red de carreteras de Pennsylvania, publicado por la universidad de Stanford. Tiene mas de un millón de nodos, que serias las intersecciones y mas de 3 millones de aristas dirigidas.

El proyecto se dividió en 3 modulos concretamente

Modulo A: Carga el dataset, construye la lista de adyacencia, hace un análisis estructural del grafo.

Modulo B: Consultas de camino mínimo punto a punto usando Dijkstra con heap mínimo y comparación con un BFS no ponderado, sobre 10 consultas P2P predefinidas en las instrucciones del proyecto

Modulo C: construcción del subgrafo inducido.

2. Descripcion del dataset.

El archivo roadNet-PA.txt se descargo desde la URL oficial:

<https://snap.stanford.edu/data/roadNet-PA.html>

Cada línea del archivo, excepto los comentarios que empiezan con #, contienen 2 enteros separados siguiendo la siguiente estructura (nodo, destino). Nosotros lo cargamos como no dirigido. Como los ids originales no son consecutivos, se tuvo que mapear usando

`unordered_map<int, int>`

2.1 Comparacion con valores oficiales de SNAP.

Estadistica	Valor SNAP publicado	Valor Obtenido
Numero de nodos	1,088,092	
Numero de aristas (noDirig)	1,541,898	
Nodos en la componente conexa principal	1,087,562	1087561
Diametro	782	592
Nodo de mayor grado	(no encontrado)	Nodo:683.267 con el id Interno (grado 9)
Grado promedio	2.83	2.83

Fig1.

Nota: durante las primeras corridas del modulo A estaba contando 3.083.796 de aristas, ya que estábamos calculando como si fuera un grafo no dirigido. Tambien el contador de aristas lo pusimos antes del if que comprueba si la línea es un comentario o es un dato, esto agregaba mas datos de los que hay.

2.2 Observaciones

Como se puede observar en la tabla de la fig.1 los datos realmente coinciden con los publicados por SNAP, se puede confirmar corriendo el programa y viendo el archivo resultados.txt.

Consideramos que el Diametro no coincide ya que no se parte desde el nodo mas alejado, sino de un nodo en especifico que es el nodo con mayor grado, un nodo que puede estar mas centrado que cualquier otro.

Tambien se observa que el nodo es bastante disperso, al ver el grado promedio de 2.83 se infiere que en promedio cada intersección esta conectada a 3 carreteras, si nos ponemos a analizar muchos cruces de las ciudades reales son intersecciones en T o en +. Por lo tanto considero que tiene sentido.

3. Diseño e implementación

3.1 Estructura del proyecto

El código esta estructurado dentro de una carpeta src/

- graph.cpp y graph.h //lista de adyacencia, básicamente construye el grafo

- loader.cpp y loader.h //parsea el archivo y configura los IDs

- bfs.cpp y bfs.h //implementa el BFS no ponderado con conteo de nodos explorados
- dijkstra.cpp y dijkstra.h //Implementa el algoritmo con heap mínimo y reconstruirCamino()
- consultas.cpp y consultas.h //10 consultas P2P solicitadas
- subgraph.cpp y subgraph.h //modulo C: subgrafo
- main.cpp / donde se corren los módulos

En git hay 5 ramas

- main // default

- feature/parser //se implemento el parser para que los del equipo trabajaran en todos los módulos, ya que es algo esencial para todo el proyecto.

- feat/BFS // se implemento el bfs solicitado en modulo A y se agregaron muchas mas funciones en el main que hacían falta.

- modulo-B //se implemento el modulo B

- modulo C//se implemento el modulo C

3.2 Estructura de datos principal

Decidimos usar `std::vector <std::vector<std::pair<int,int>>>`; donde el índice del vector es el ID que le asigna y cada entrada es un vector de pares (vecino, peso)

Sobra decir que no se uso matriz de adyacencia ya que es muchísimo mas costosa que hacer la lista, definitivamente para estos grafos tan grandes no es una opción viable.

3.3 Asignacion de pesos

El dataset original no tiene pesos en las aristas, si se quiere lograr un nivel de realismo más grande cada peso dependería de factores como la longitud del tramo, tiempo de viaje, tipo de via, entre otros. Se hizo caso a lo que dice el enunciado de usar la semilla `srand(42)` y generar valores aleatorios de 1 a 10.

4. Analisis de complejidad

4.1 BFS $O(V+E)$

BFS recorre cada nodo una vez y por cada nodo visitado examina a los vecinos. Para grafos no dirigidos sería $2E$ ya que las consultas sobre los vecinos se haría 2 veces, por lo tanto se representaría como $O(V + E)$

En pruebas realizadas por nosotros mismo el tiempo tomado fue de: **1627ms**

4.2 Dijkstra con heap mínimo

Este algoritmo tiene complejidad $O((V+E)\log V)$

Se extrae cada nodo del heap como máximo una vez, también cada arista provoca como máximo una inserción nueva en el heap. Estas 2 afirmaciones nos dan un resultado de $O((V+E)\log V)$.

5. Resultados de consultas P2P

Consulta	Origen	Destino	Dist Dijkstra	Salto BFS	Nodos Dijkstra	Nodos BFS	T(ms) Dijkstra	tBFS (ms)
Q01	1	500.000	491	101	1.087.561	1.087.561	314,70	96,06
Q02	100	1.000.000	2.294	484	1.087.561	1.087.561	343,94	76,73
Q03	50.000	750.000	2.642	545	1.087.561	1.087.561	274,70	63,43
Q04	200.000	800.000	624	124	1.087.561	1.087.561	299,88	76,10
Q05	300.000	100.000	3.209	666	1.087.561	1.087.561	275,02	96,16
Q06	1	1.087.562	422	95	1.087.561	1.087.561	516,82	110,7
Q07	500.000	1	491	101	1.087.561	1.087.561	502,28	131,5
Q08	250.000	600.000	1.485	300	1.087.561	1.087.561	645,47	143
Q09	10.000	900.000	871	169	1.087.561	1.087.561	413,04	76,3
Q10	400.000	150.000	778	167	1.087.561	1.087.561	285,81	57,41

5.1 Analisis Dijkstra vs saltos BFS

Al dividir la distancia de dijkstra por los saltos BFS obtenemos un valor cercano a 5,5 en casi todas las consultas, tiene sentido al saber que los pesos están limitados por $\text{rand}()\%10+1$, entonces el peso promedio por arista es de 5,5.

La diferencia esta en que el camino que encuentra Dijkstra no es necesariamente el mismo que BFS, lo que hace el BFS es minimizar saltos, a este no le importa el peso, en cambio Dijkstra minimiza la suma de pesos, esta es la diferencia de esos 2 algoritmos. Se analizo

que Dijkstra es un algoritmo mas lento que el BFS, es hasta 4 veces mas lento, esto por las operaciones heap

Observe que la consulta Q01 y Q07 los valores son idénticos, esto es porque una es la consulta inversa de la otra, esto sucede porque el grafo es no dirigido, entonces el camino de A hasta B es lo mismo que B hasta A.

6. Analisis del subgrafo

A partir de los caminos construidos de Q01 y Q06, construimos el subgrafo inducido tomando la unión de los nodos de ambos caminos.

6.1 Como se construye el subgrafo

Sabemos que los caminos comparten origen, también pasan por algunos nodos en común. Entonces, tomamos todos los nodos del camino Q01 mas los del Q06, se eliminan los duplicados `set<int>`, luego recorre las aristas y conserva las aristas que sus extremos estén en el conjunto de los nodos del subgrafo.

6.2 Resultados subgrafo

estadistica	Valor
Nodos del subgrafo	9
Aristas del subgrafo	9
Peso total MST	235
DAG?	NO

6.5 Interpretacion del subgrafo

El subgrafo inducido por los caminos `Q01 y Q06` si se representa en la vida real serian 2 rutas que parten desde el mismo origen hacia destinos diferentes en otros puntos de Pennsylvania. Ambos caminos comparten los mismos nodos al principio mentalmente se hace una Y antes que separen sus caminos, este juego mental es maravilloso, nos gustaría en un futuro graficar estos algoritmos para lograr un mayor entendimiento del observador.

7. Conclusiones

Este proyecto para nosotros fue un gran reto porque veníamos trabajando con grafos muy pequeños comparados con el de este proyecto. Realmente deja varias enseñanzas para futuros grandes proyectos:

- 7.1 El tamaño cambia las decisiones: Cuando un grafo tiene 10 nodos no importa si usas matriz o una lista, pero con 1 millón de nodos cambia totalmente la cosa, lo que el profe insistía en clase, puede funcionar con 50 elementos, pero a la hora de enfrentarse a grafos de verdad es donde hay que sacar los fundamentos y tomar la decisión correcta.
- 7.2 El uso de `srand(42)` para los pesos garantiza que cualquier persona que clone el repo y compile el código obtenga exactamente nuestros mismos resultados. Esto era algo que en proyectos anteriores no le habíamos dado tanta importancia, pero acá fue clave para poder verificar consultas entre equipos.
- 7.3 El trabajo en git es sumamente importante: llevar un orden en git da un seguimiento mas profundo al proyecto, muy interesante tener todo organizado por ramas y poder ver que feat agregan los compañeros para entender mejor que hizo exactamente

8. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. (Capítulos 22, 23 y 24: BFS, MST, Dijkstra.)
- Material del curso EDA <https://github.com/carlosalvarezh/EstructuraDatosAlgoritmos1>
- SNAP Pennsylvania Road Network: <https://snap.stanford.edu/data/roadNet-PA.html>

9. Herramientas utilizadas

Para la practica se utilizó a Claude de Anthropic con los siguientes fines

Aclaración de conceptos y depuración: Inicialmente no compilaba el programa, pero no fue por problema de lógica, sino por problema del linker, resulta que el .exe no se creaba. Todo era culpa del CMAKE, de la mano de Claude se detectó este problema y darle solución.

Se le pidió a Claude aclaración de conceptos de OOP, ya que habían muchas cosas que no recordaba o que simplemente no sabía hacer en C++, es decir, nos ayudó con la sintaxis en gran medida.

Verificación: Todo el código fue escrito, leído y entendido por los integrantes del equipo.

Cualquier sugerencia de la IA fue probada localmente, ajustada al estilo del resto del proyecto y validada contra los valores publicados por SNAP cuando aplicaba. En caso de revisión oral, cada integrante puede explicar los módulos en los que trabajó:

Emmanuel Montoya: parser/loader y módulo A (BFS para diámetro y estadísticas estructurales).

Jairo Ariza: módulo B (Dijkstra con heap mínimo, BFS no ponderado, ejecución de las 10 consultas P2P y reconstrucción de caminos).

Miguel Álvarez: módulo C (subgrafo inducido, Kruskal para MST y verificación DAG con DFS).